KAFKA

Yet to cover below

Streaming API

Load balancing

Replication

Retention Period

Retry Strategy

Exactly One Message

Kafka Performance Tuning

Kafka Monitoring

How to start Kafka Cluster?

Start Zookeeper with Zookeeper. Properties

Start KAFKA-server.start.sh with server. Properties

Can you run Kafka without zoo keeper?

Yes, KAFKA 2.8.0 has **Kraft Metadata Mode** where servers from KAFKA Cluster Works as **Quorum** for Managing all Meta data.

Same server can handle Broker functionality also,

will have leader follower concept for high availability.

How do you add broker in cluster?

Kafka server.properties has broker.id=0

```
Start Kafka server using
bin/kafka-server-start.sh config/server1.properties

Start Kafka server using
bin/kafka-server-start.sh config/server2.properties

Start Kafka server using
bin/kafka-server-start.sh config/server3.properties
```

How to create a Topic?

```
bin/kafka-topics.sh
--create
--zookeeper localhost:2181 ¥
--replication-factor 1 ¥
--partitions 1 ¥
--topic textTopic
```

How to create Consumer Group?

Create a 1st Consumer with Group name and Then 2nd 3rd Use same Group name

```
kafka-console-consumer.sh
--bootstrap-server localhost:9092
--topic firstTopic
--group my-first-application
```

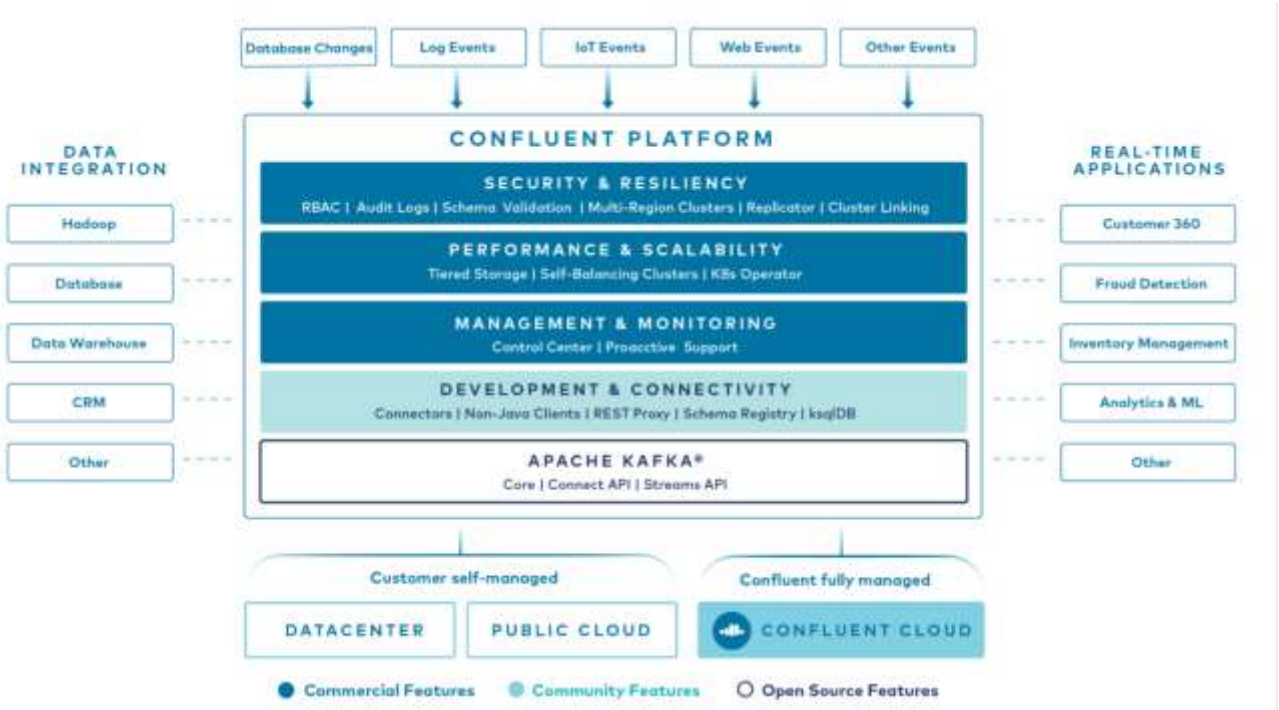
What is Boot Strap Server?

Starting point to make connection to Kafka

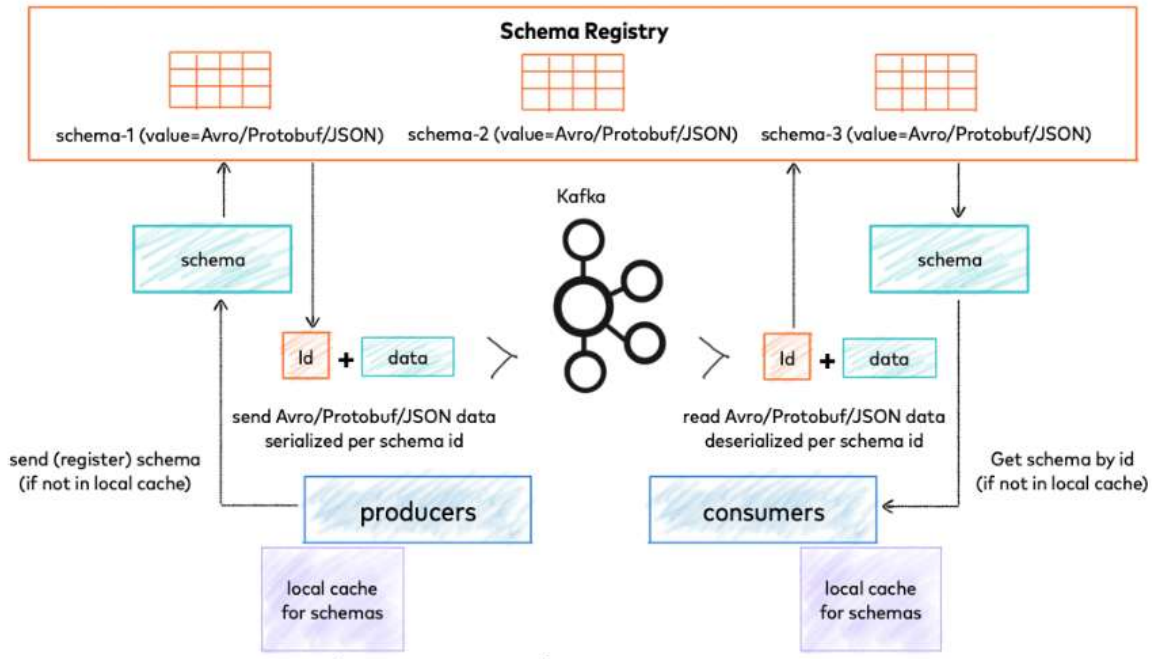
Holds Info about all the broker of cluster

Select More than one Broker server as Boot Strap servers.

Apache Kafka Vs Confluent Kafka



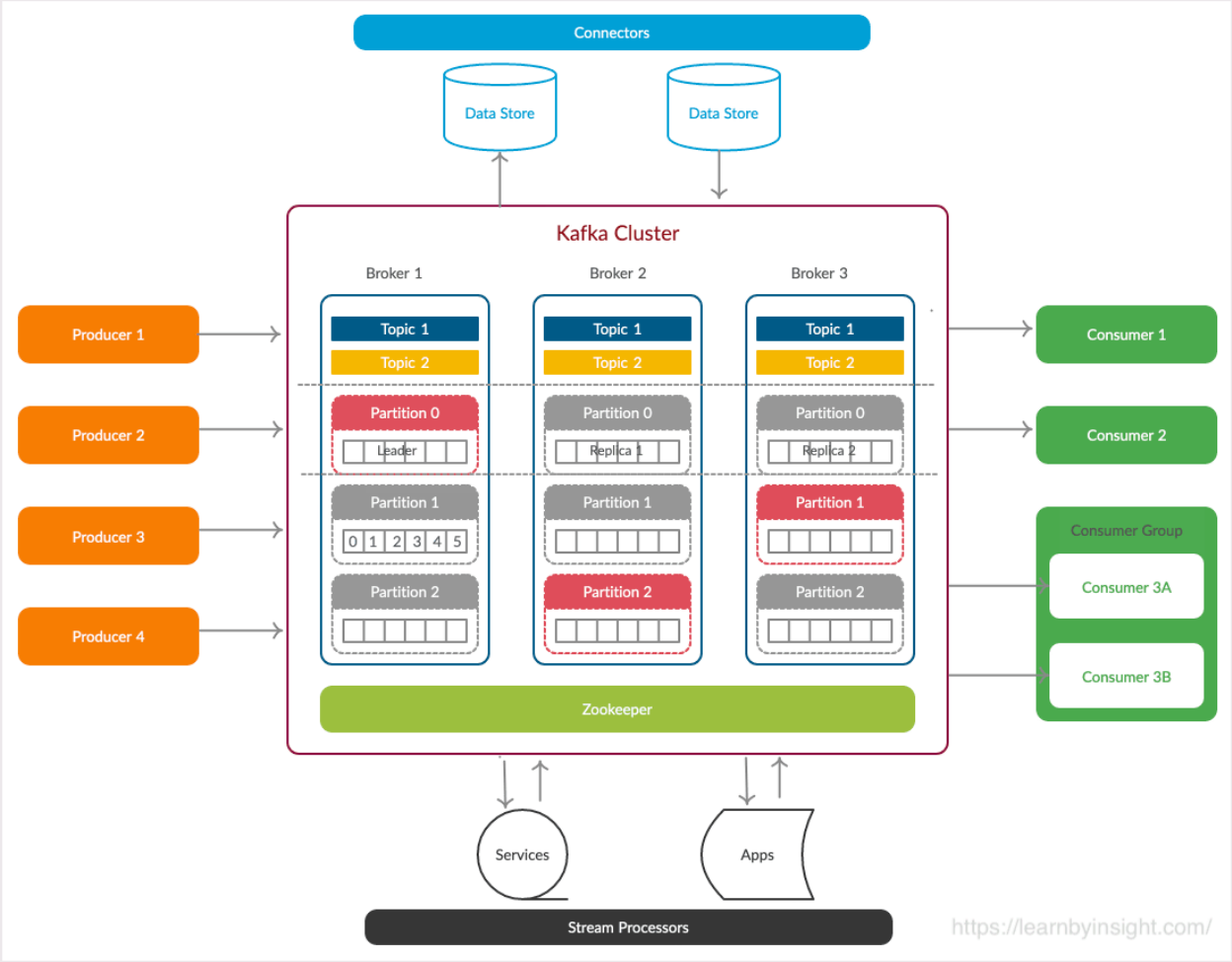
What is Schema Registry?



Confluent Schema Registry for storing and retrieving schemas

How Many Schema Registry?

Kafka Structure



How to decide how many partitions to be created?

Max 2Lakh Partitions, Max 4K partition per broker

Depends on **Throughput** you need, if you want 100Mbps and Producer can produce only 10Mbps then you need 10 Partitions

A partition is a unit of parallelism

Depends on How many Consumers You want to create, because Consumers can be less then or equal to No of Partition **No Of partition >= Consumer**

Disadvantage of Many Partitions:

Many **Open File handler** opened by broker, there is OS limit of Number of Open file handler

Replication across Broker will slow down as need to sync Many partitions across Broker

Zookeeper need to **elect partition Leader** again and again which will take time / Solved in Quorum Controller based cluster setup

How to choose Replication Factor?

At least 2 and a maximum of 4. The recommended number is 3 as it provides the right balance between performance and fault tolerance,

3-1 = 2 broker failing is covered

Disadvantage of high Replication Factor

High latency by producer if Ack=all

More Disk space utilization

Does Kafka have Visibility Time out as in AWS SQS?

NO, Kafka do not have **competing consumer** situation itself, **Only One consumer** is assigned to one partition.

Such condition does only arise when two or more consumers are trying to consume same message.

So, No need to hide message from other consumers while one is processing

How to decide which message Goes on Which Partition?

Each **partition is a single log file** where records are written to it in an append-only fashion.

Producer Uses **Partition Key** to keep Related Message on one partition

Default Partitioner, Round robin Partitioner or **Murmur Partitioner**

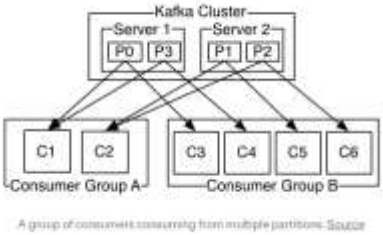
Client can create **Custom Partitioner** as per business logic.

How to force a consumer to read a specific partition in Kafka?

```
consumer.assign(Collections.singleton(new TopicPartition(TOPIC_NAME, 1)));
```

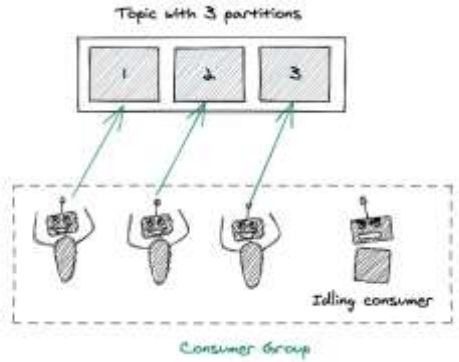
What is Consumer Group?

Group of Consumers instance Subscribing to One topic, One Offset One Partition for One Consumer Group.



6 partition 2 Consumers in group each consumer will pic Message from 3 partition

3 partition and 5 consumers then 2 consumers will be unused.

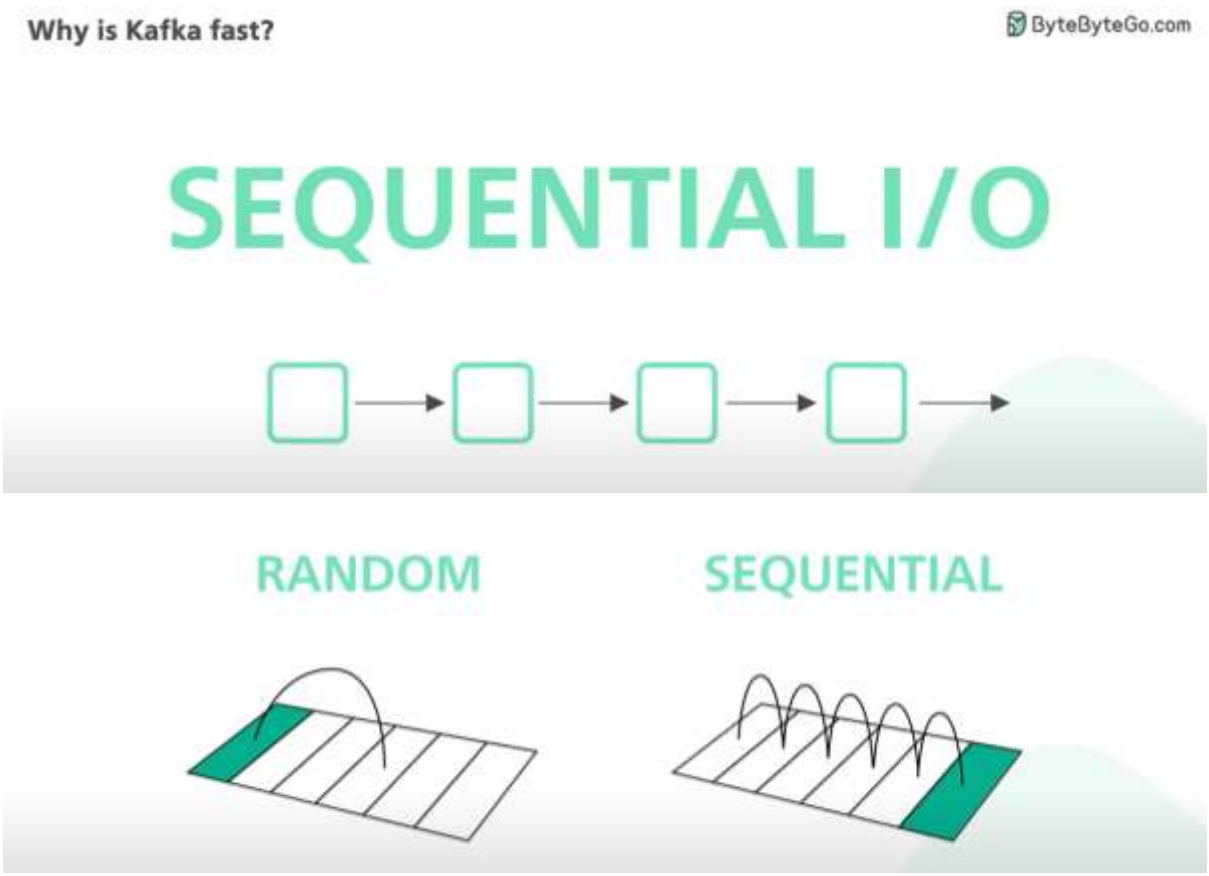


Why Kafka is fast?

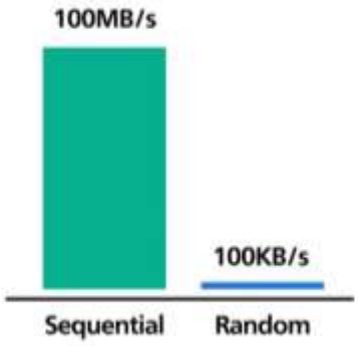
https://www.geeksforgeeks.org/why-apache-kafka-is-so-fast/

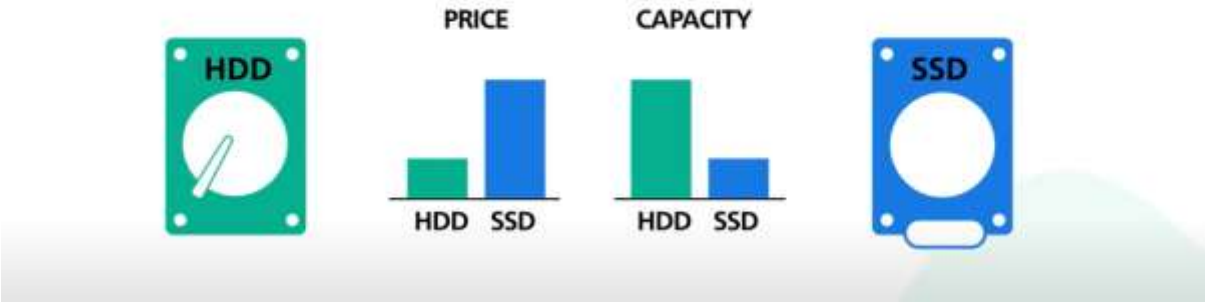
1. Low-Latency I/O: There are two possible places which can be used for storing and caching the data: **Random Access Memory (RAM)** and **Disk**.

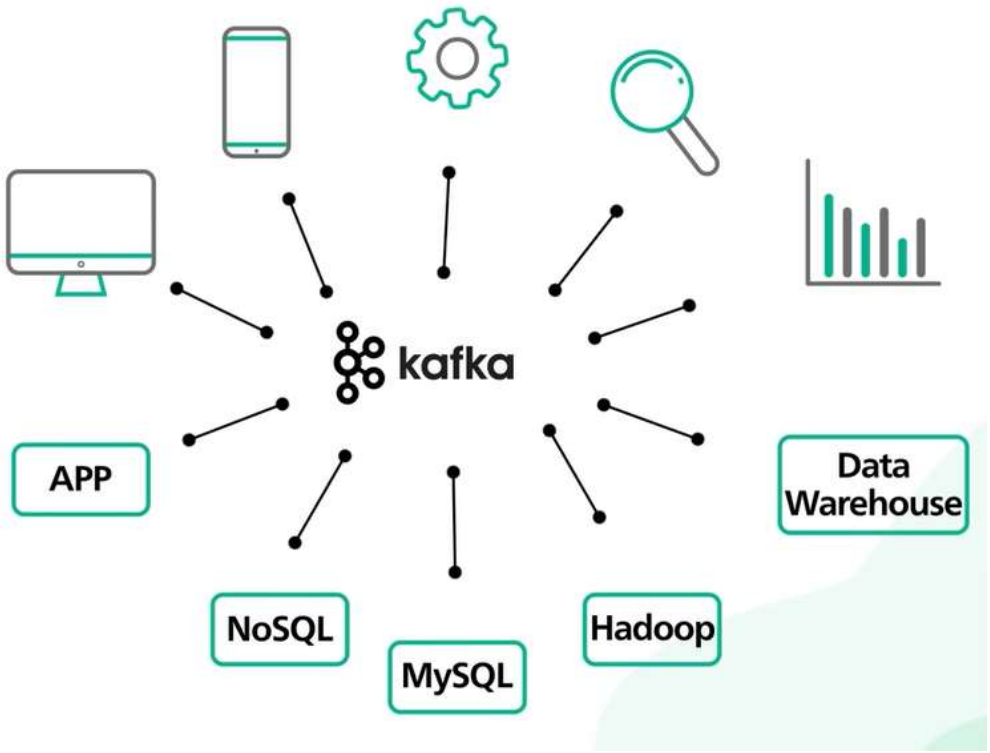
- An orthodox way to achieve low latency while delivering messages is to use the RAM. It's preferred over the disk because disks have high seek-time, thus making them slower.
- The downside of this approach is that it can be expensive to use the RAM when the data flowing through your system is around 10 to 500 GB per second or even more.



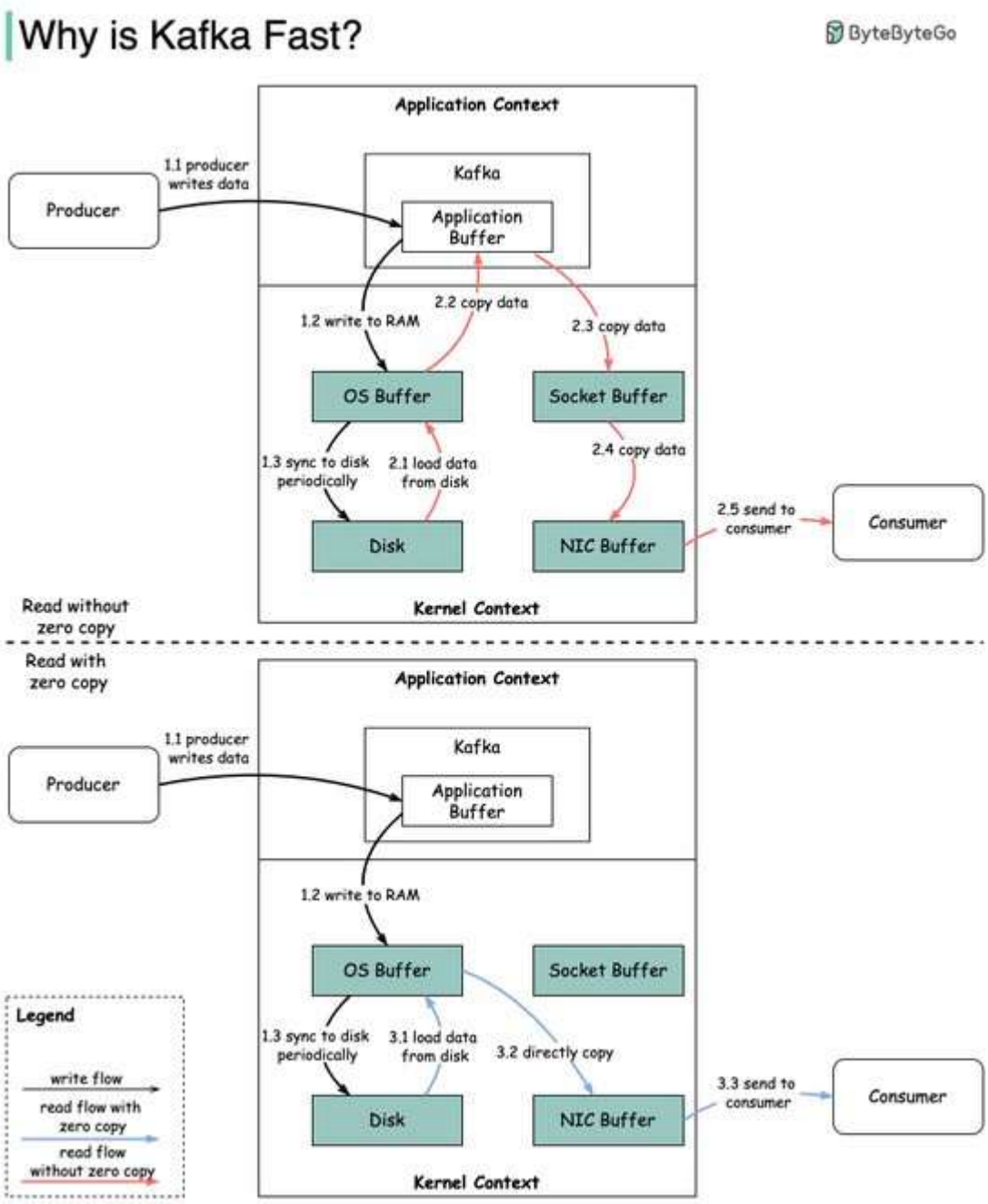
Append Only rule logs







Read with Zero Copy



Modern network card has DMA Direct Memory Access Where CPU is not involved while accessing Memory.

4. Optimal Data Structure: Tree vs. Queue: Kafka uses Queue

5. Horizontal Scaling:

6. Compression & Batching of Data: reduce network calls

Why can't I use **DB table** Instead of Kafka?

High Throughput **10-100GB of data per second** DB cannot handle such Scale

Multiple consumers will try to access same table will **lock table.**

Slower performance as each time record needs to be selected, even if indexed.

Kafka is **Hardware Optimized** Append Only log files Which has Faster Producer latency

Resilience will suffer as DB is down Messaging will be down.

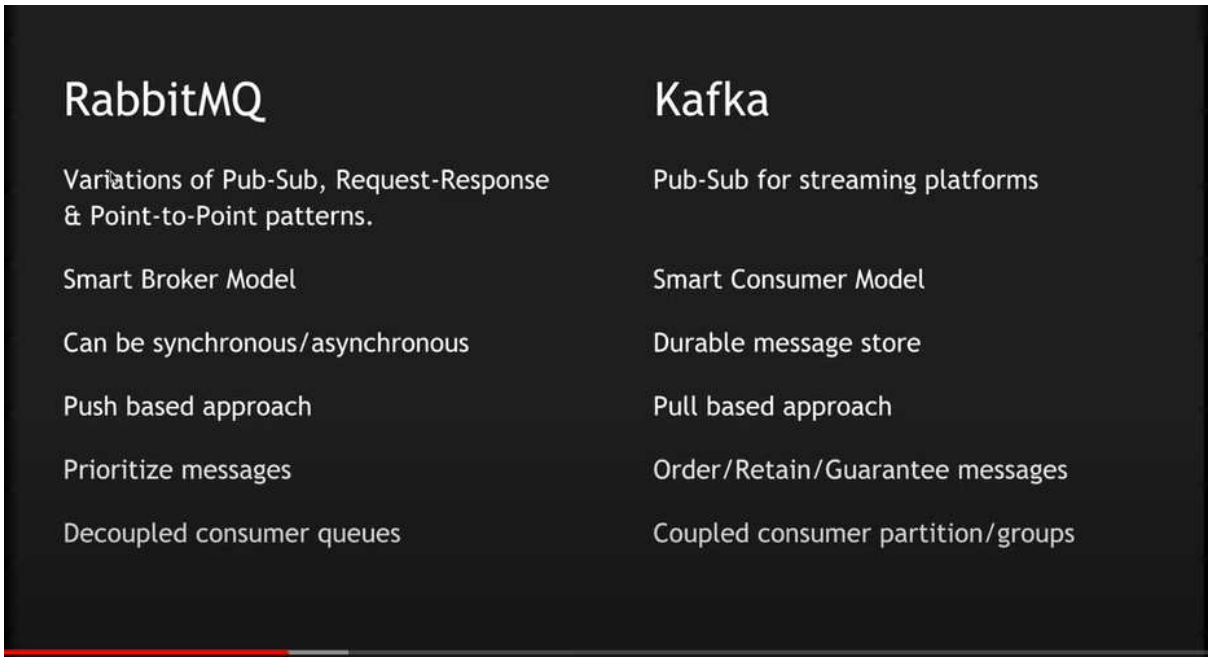
No replications

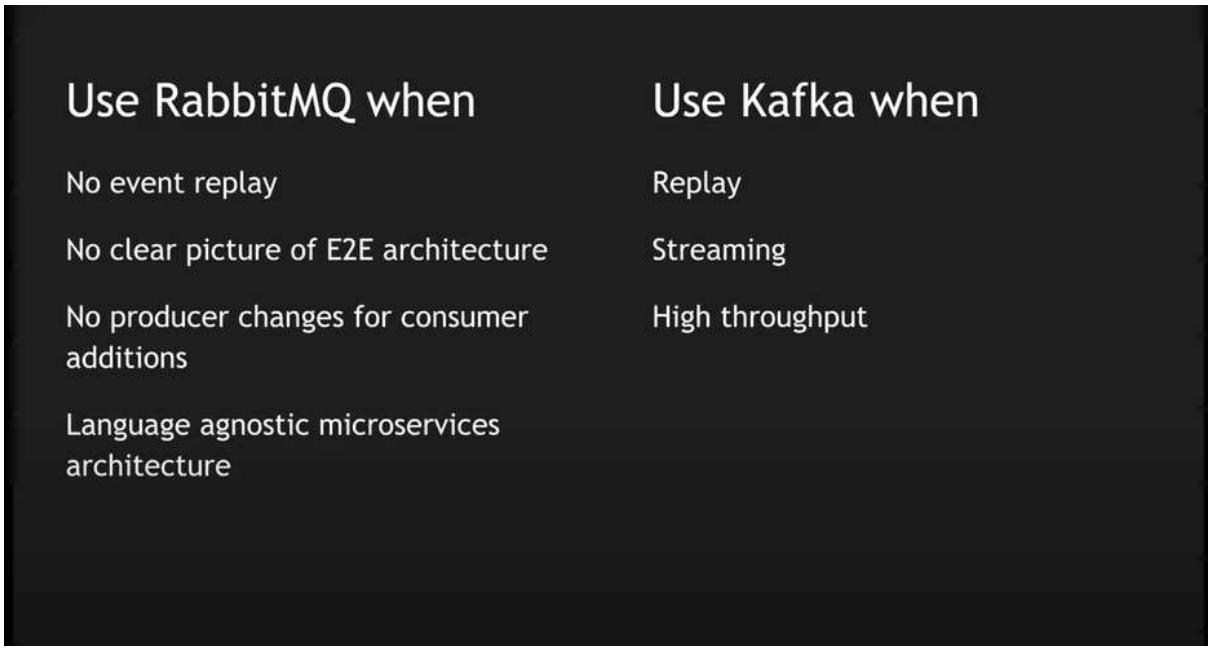
Horizontal Scaling of DB is not possible, many connections will slow down one DB

KAFKA	JMS
Don't wait for Acknowledgement from Broker	Wait for Confirmation of write
Ordered Per Partition	No Order Guaranty
No filter at Consumer, Consumer App have to filter it after read.	"JMS Select" for Filtering
Pull Type	Push Type
No Performance Hit by adding Consumers	Slows Down with More Consumers
Replication	No Replication
Message Can be re-read	Can't Re-read Message
Reactive	Imperative

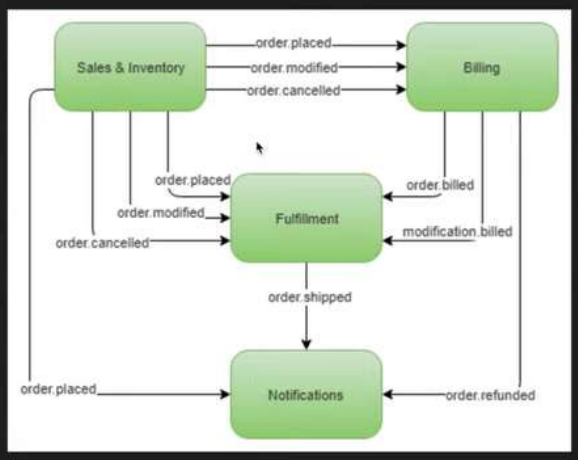
Kafka	Rabbit MQ
Pub Sub	Queue
Message Not removed After Consumption	Message Removed after Consumption
Replay	No Replay

Topic	Queue


RabbitMQ
Variations of Pub-Sub, Request-Response & Point-to-Point patterns.
Smart Broker Model
Can be synchronous/asynchronous
Push based approach
Prioritize messages
Decoupled consumer queues
Kafka
Pub-Sub for streaming platforms
Smart Consumer Model
Durable message store
Pull based approach
Order/Retain/Guarantee messages
Coupled consumer partition/groups

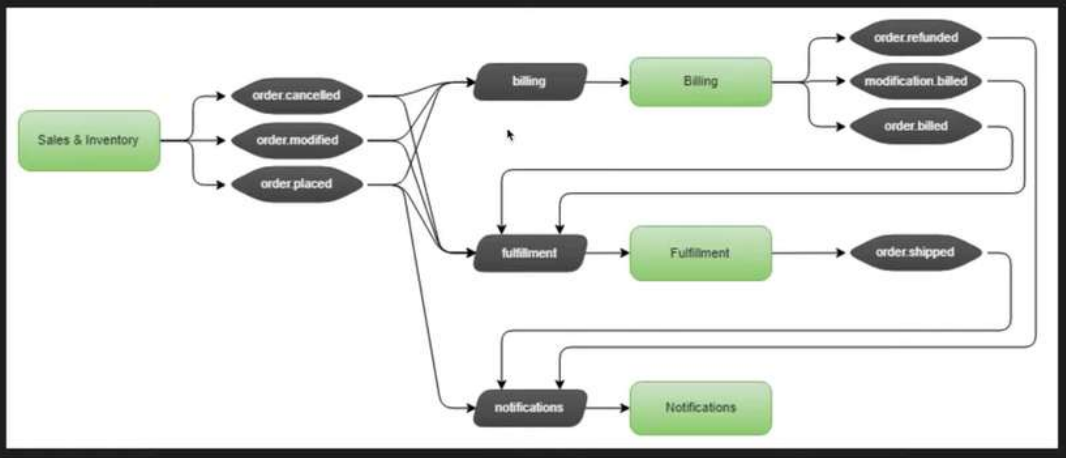

Use RabbitMQ when
No event replay
No clear picture of E2E architecture
No producer changes for consumer additions
Language agnostic microservices architecture
Use Kafka when
Replay
Streaming
High throughput

Sample Case study



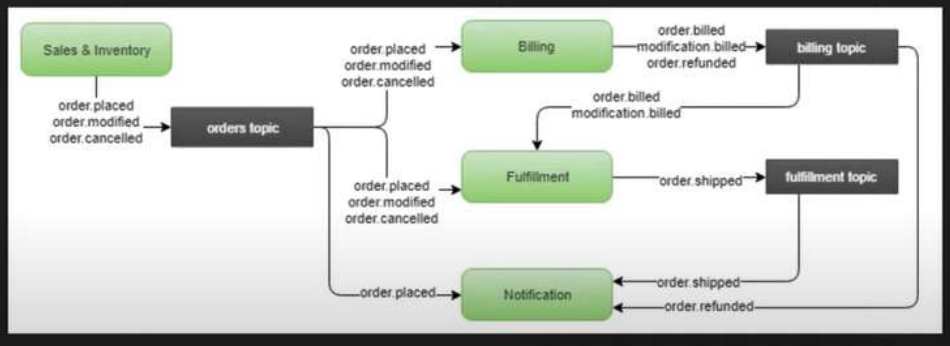
Source: https://jack-vanlightly.com/blog/2018/5/21/event-driven-architectures-queue-vs-log-case-study

Sample Case study: RabbitMQ



Source: https://jack-vanlightly.com/blog/2018/5/21/event-driven-architectures-queue-vs-log-case-study

Sample Case study: Kafka (option 2)



Source: https://jack-vanlightly.com/blog/2018/5/21/event-driven-architectures-queue-vs-log-case-study

How to publish JSON messages on Apache Kafka?

https://www.geeksforgeeks.org/spring-boot-how-to-publish-json-messages-on-apache-kafka/?ref=lbp

```
@Bean
   public ProducerFactory<String, Student>{
         return DefaultKafkaProducerFactory<>(config);//--config  Serializer
   }

@Bean
   public KafkaTemplate<String, Student> kafkaTemplate()
  {
     return new KafkaTemplate<>(producerFactory());
  }


kafkaTemplate.send( TOPIC, new Student());
```

How to consume JSON messages using Apache Kafka?

https://www.geeksforgeeks.org/spring-boot-how-to-consume-json-messages-using-apache-kafka/?ref=lbp

```
@Bean
public ConsumerFactory<String, Student>
studentConsumer(){
return new DefaultKafkaConsumerFactory<>(
configMap,
new StringDeserializer(),
          new JsonDeserializer<>(Student.class));
}

@Bean
public ConcurrentKafkaListenerContainerFactory<String,Student>
   studentListner()
```

```
    {
        ConcurrentKafkaListenerContainerFactory<String,Student>
            factory
            = new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(studentConsumer());
        return factory;
    }

@KafkaListener(topics = "JsonTopic",
                groupId = "id", containerFactory
                            = "studentListner")
    public void
    publish(Student student)
    {
        System.out.println("New Entry: "
                        + student);
    }
```

What You define in Config?

BootStrapServers

Consumer Group

Key Serializer

Value Serializer

Key deserializer

Value Deserialiser

How to Install and Run Apache Kafka on Windows?

Which protocol Kafka uses?

Kafka uses **a binary protocol over TCP**

What if Kafka consumer goes down?

If the consumer crashes or is shut down,
its partitions will be re-assigned to another member,
which will begin consumption from the last committed offset of each partition.
If the consumer crashes before any offset have been committed then,
the consumer which takes over its partitions will use the reset policy.

What port does Kafka use?

Service	Servers	Default Port
Kafka	**Kafka Server**	**9092**

What is the max message size in Kafka?

1MB

The Kafka max message size is **1MB**. In this lesson we will look at two approaches for handling larger messages in Kafka. Kafka has a default limit of 1MB per message in the topic.

How to send Large Messages in Apache Kafka?

https://www.conduktor.io/kafka/how-to-send-large-messages-in-apache-kafka

Option 1: using an external store (GB-size messages)

Option 2: sending larger messages to Kafka (ex less than 10MB but over 1MB)

Here we need to modify topic, producer and consumer configurations to allow for a bigger message size.

The broker-side setting is message.max.bytesand the topic-side setting is max.message.bytes.

What is offset and partition in Kafka?

Kafka maintains a numerical offset for each record in a partition.

This offset acts as a unique identifier of a record within that partition, and also denotes the position of the consumer in the partition.

How is offset generated in Kafka?

For all consecutive messages within a segment, the offset of a message can be computed by its logical position within the segment (including the offset of the first messages).

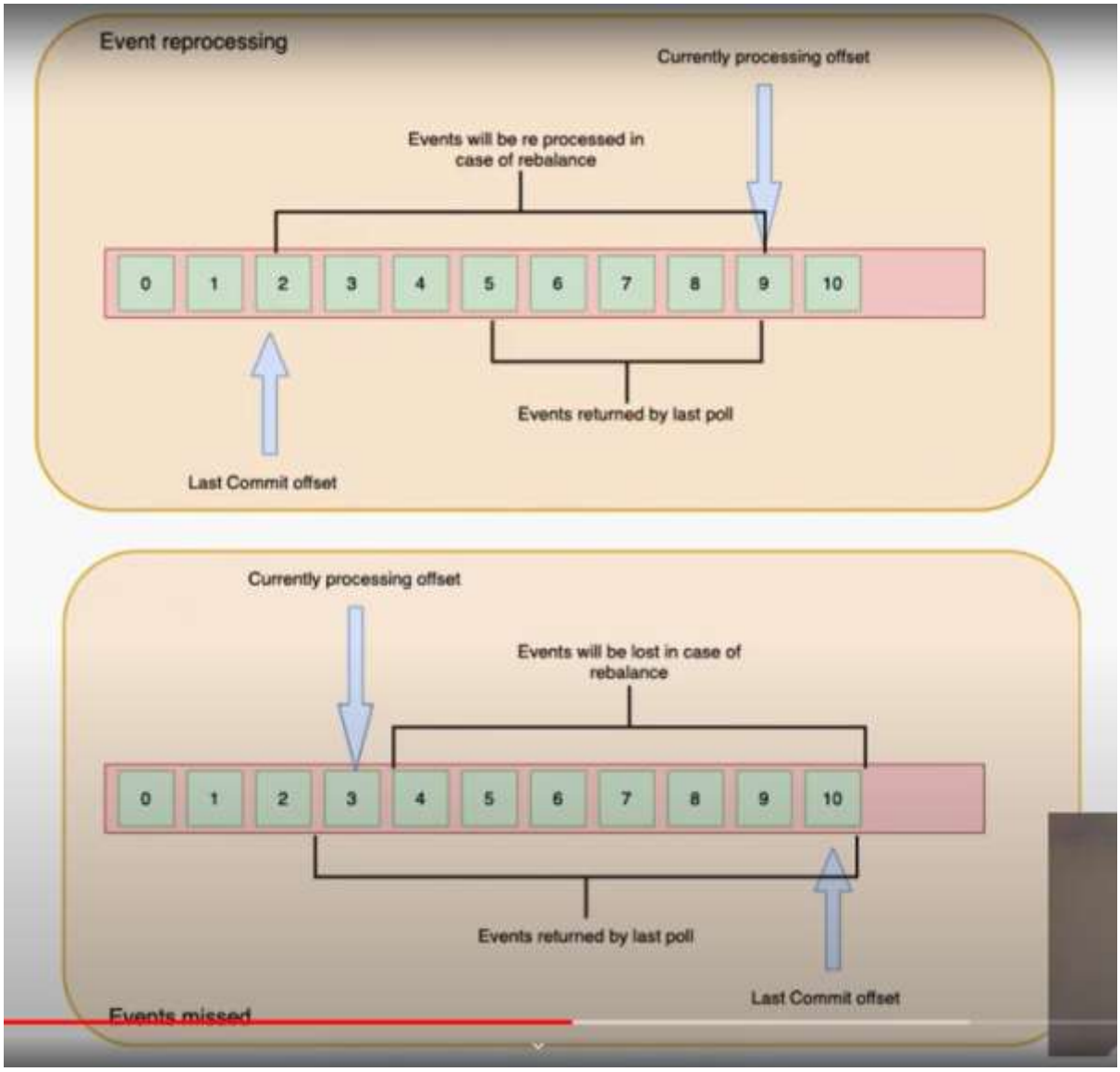
If you start a new topic or actually a new partition, a first segment is generated and its start offset zero is inserted into the index

Who maintains the offset in Kafka?

https://www.youtube.com/watch?v=KOu6DVdaY24&ab_channel=TechWithAman

Offset Topic contains information about which partition's which group is at which offset

For Producer and For Consumer _consumer_offsets



Consumer Offsets

- What is consumer offsets.
- Consumer offset are stored in special topic **__consumer_offsets**
- Its responsibility of consumer to produce offset message to above topic.
- After rebalance new consumer pick current offset from this topic.
- We can reprocess or miss messages.
- Types of commits
  - Auto commit
  - Manual commit

Types of commits

- Auto commit
- Manual commit
  - Sync commit.
  - Async commit

How Kafka implements exactly once?

processing.guarantee=exactly_once_v2 in StreamsConfig

isolation.level=read_comitted

Every message is marked with if it's committed or not.

read_committed Broker will show only **committed** records to any consumer.

Consider case of transfer 10Rs between A->B

Steps are:

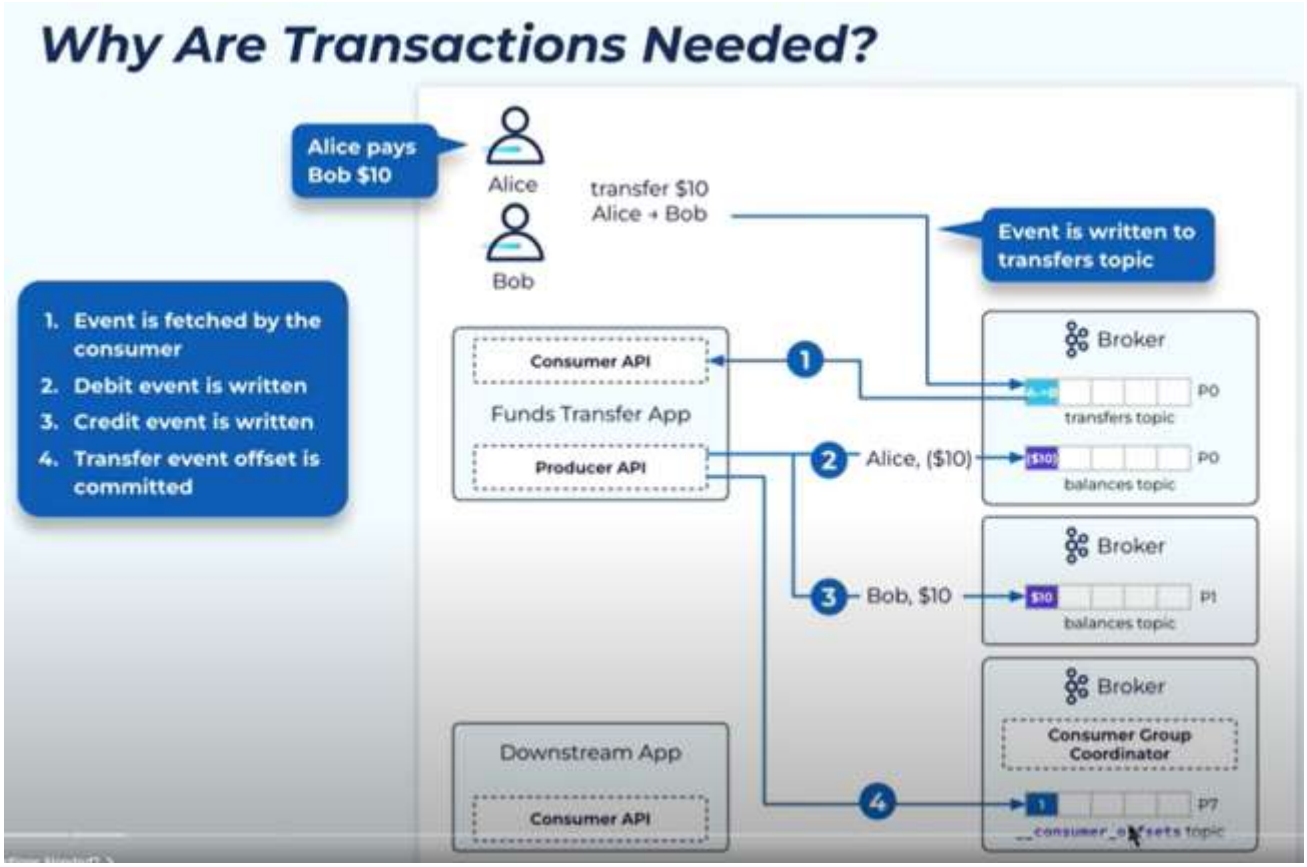
1. Put message on P0 for Debit from A
2. Put message on P1 for Credit to B
3. Put message to consumer_offsets topic

Transactions ensures that all above 3 steps are done Atomically, **all executed or None executed.**

Concept:

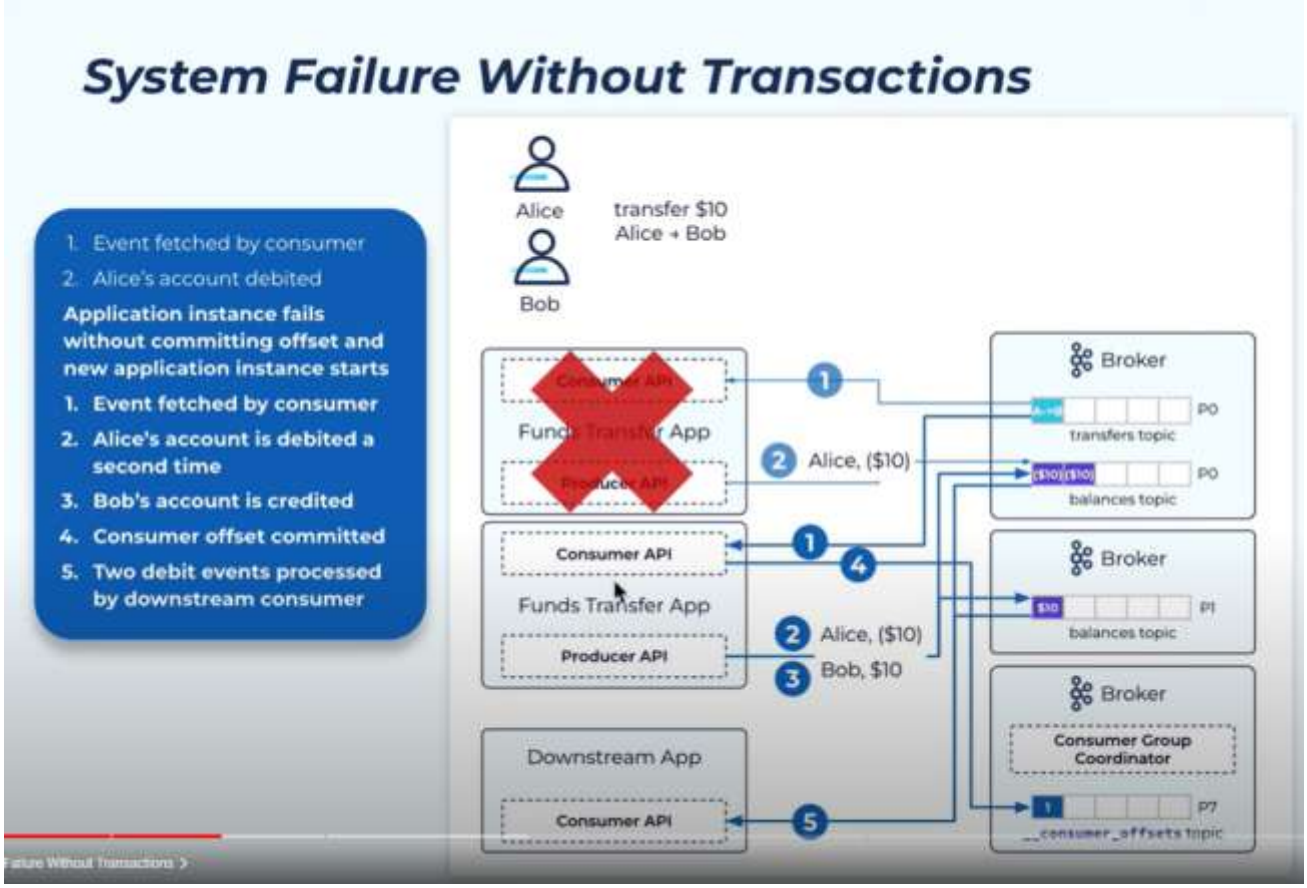
Kafka has **append-only** concept, it does not delete or modify messages but, it can say Broker That Please ignore the message which was written by uncommitted transactions.

Transaction Coordinator: third person (Broker) who supervise the whole transaction and will commit once all steps are completed and then only consumer can see that records. as we set **isolation.level=read_comitted**



Steps without Transaction Manager:

1. Fund Transfer App will read with Consumer API (got Order to transfer 10Rs in Account B from A)
2. Producer API will send Message → Debit 10Rs from account A on P0
3. Producer API will send Message → Credit 10Rs to account B on P1
4. Producer API will send Message → I am done for P0's offset 1, send Message to __consumer_offset topic on P7

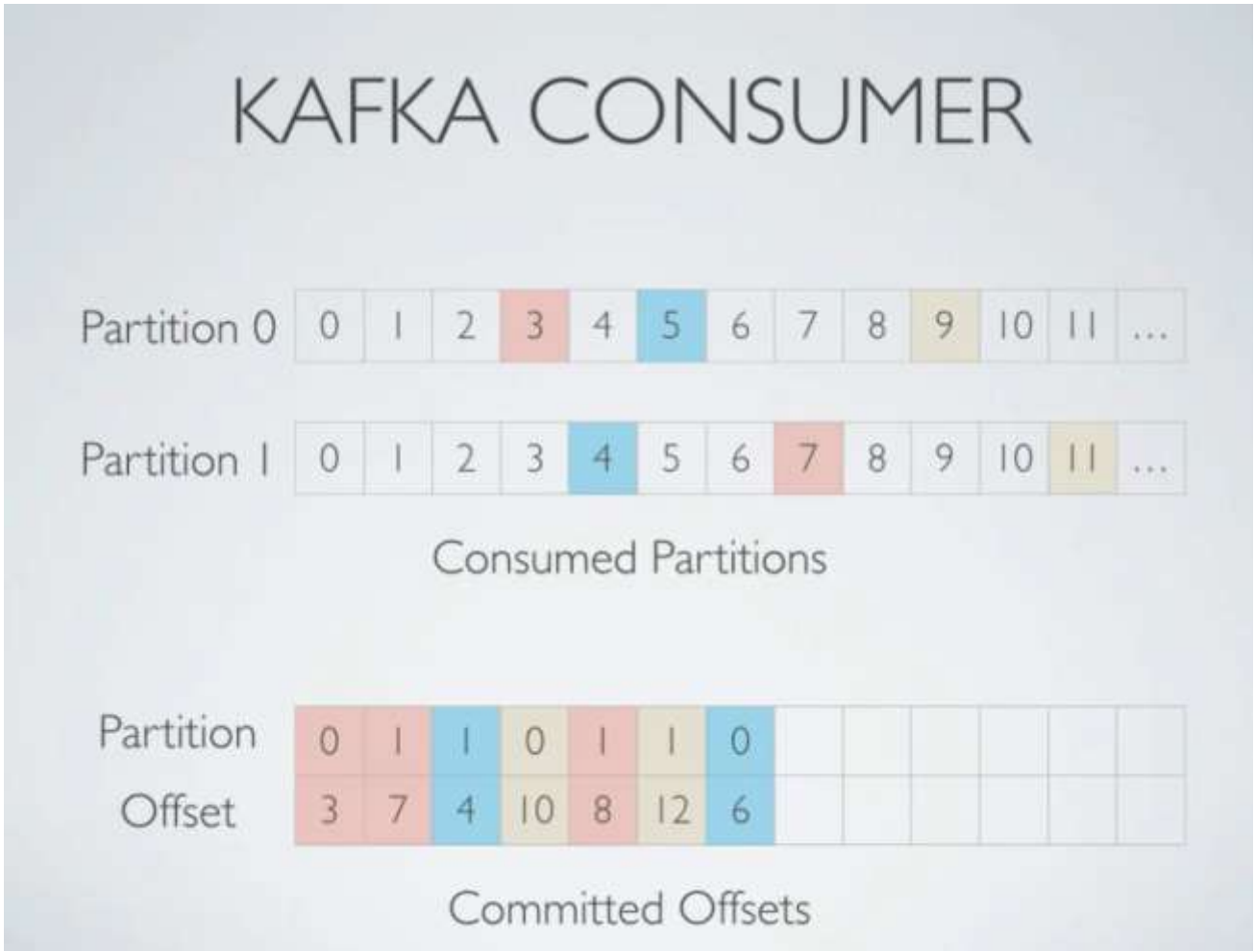


Steps When producer fails after Step 2. → after debiting from A

1. Fund Transfer App will read with Consumer API (got Order to transfer 10Rs in Account B from A)
2. Producer API will send Message → Debit 10Rs from account A on P0
3. Fund Transfer App Fails
4. New Fund Transfer App is created (Consumer + Producer both)
5. Consumer Looks in __consumer_offset topic, No Offset found for P0
6. Read P0 again, Got Transfer order (A→B 10 Rs)
7. Producer API will send Message → Debit 10Rs from account A on P0
   Here is a Problem, we already sent message to debit from A's account, Message is duplicated, A will get debited 20Rs.
8. Producer API will send Message → Credit 10Rs to account B on P1
9. Producer API will send Message → I am done for P0's offset 1, send Message to __consumer_offset topic on P7

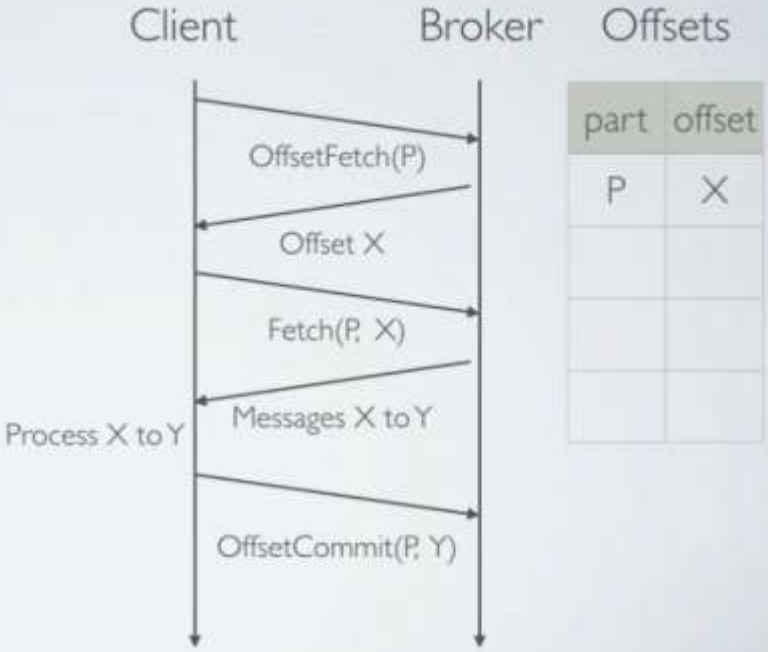
https://youtu.be/WscozkoXLHM

https://youtu.be/WscozkoXLHM?t=1759



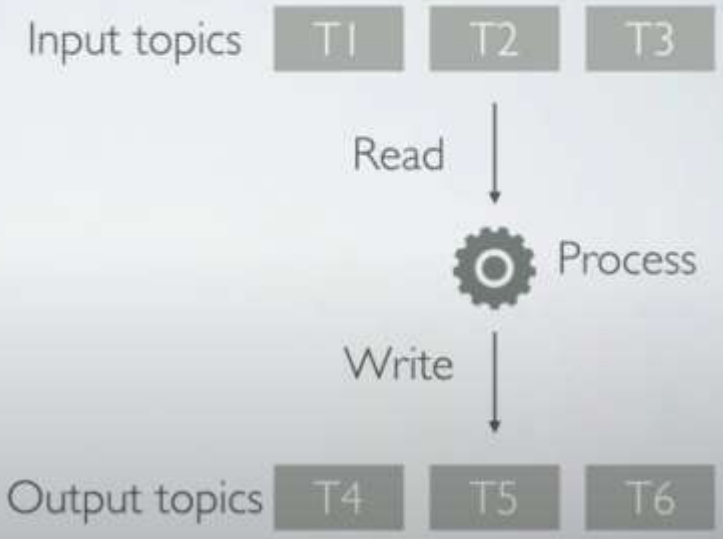
CONSUMER PROTOCOL

- Fetch API: fetch a range of records at a given offset
- OffsetCommit API: read/write offsets



PROCESSING MODEL

- Failures Types
  - Crash
  - Zombie



PROCESSING SEMANTICS

- *At-least-once*: all input is processed and the result is reflected at least once in the output
- *At-most-once*: kind of weird
- *Exactly-once*: what you really want

AT-LEAST-ONCE SEMANTICS

```
while (true) {
  data = c.poll()
  p.send(data)
  c.commitOffsets()
}
```

For the following examples, we just copy the input to the output

AT-LEAST-ONCE SEMANTICS

- Duplicates come from:
  - Producer retries
  - Failure before committing offsets
  - Evil zombies

AT-LEAST-ONCE SEMANTICS


```
while (true) {
  data = c.poll()
  p.send(data)
  c.commitOffsets()
}
```

```
while (true) {
  data = c.poll()
  p.send(data)
  c.commitOffsets()
}
```

AT-MOST-ONCE SEMANTICS

(IF YOU'RE INTO THAT)

- Message loss comes from:
  - Producer retries not permitted
  - Failure before writing data
- Zombies can cause out of order messages

SEMANTIC WEAKNESSES

- Producer retries are not safe
- Processed data is not written atomically with corresponding offsets
- No protection from evil zombies

IDEMPOTENT PRODUCER

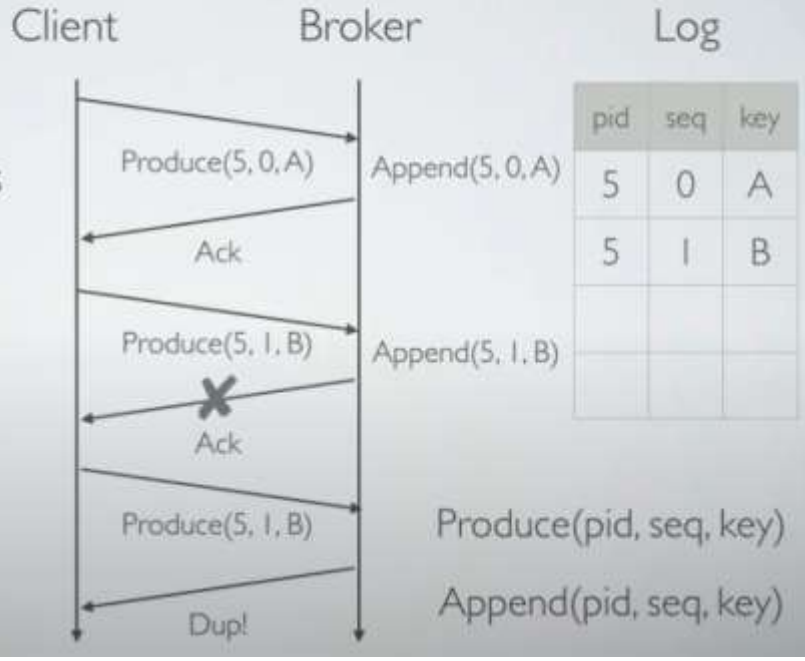
- ProducerId identifies producer instance
- Persistent sequence numbers used to deduplicate
- enable.idempotence=true

Log Message

Offset
Key
Value
Timestamp
Headers
ProducerId
Sequence

IDEMPOTENT PRODUCER

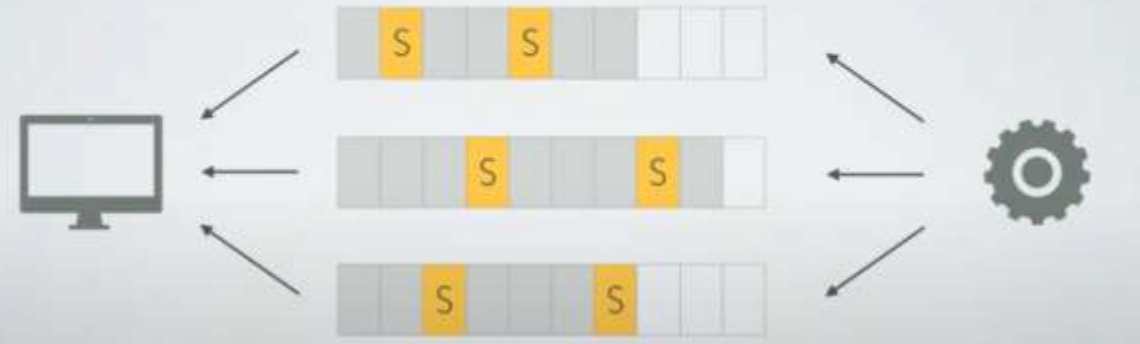
- ProducerId identifies producer instance
- Persistent sequence numbers used to deduplicate



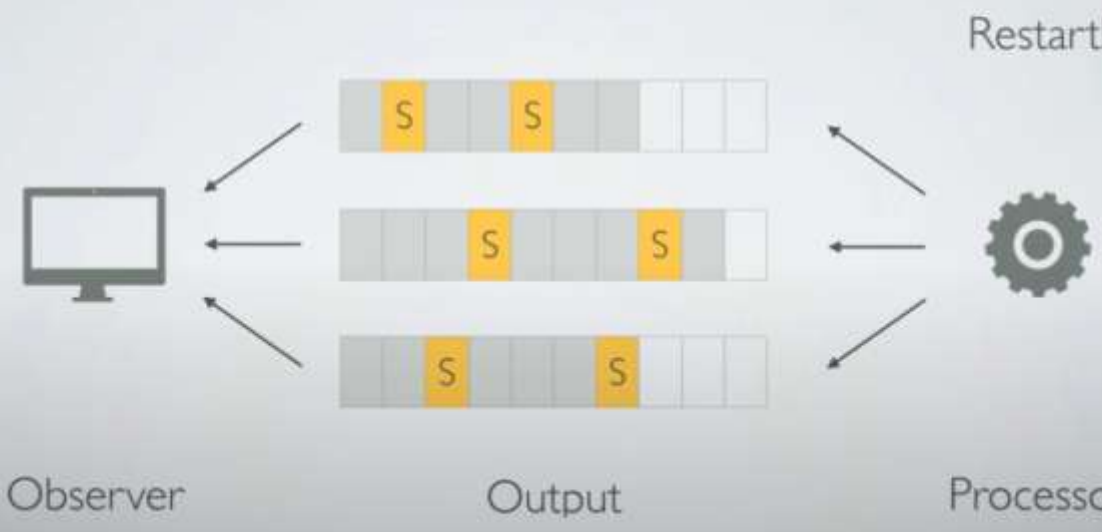
SEMANTIC WEAKNESSES

- ~~Producer retries are not safe~~
- Processed data is not written atomically with corresponding offsets
- No protection from evil zombies

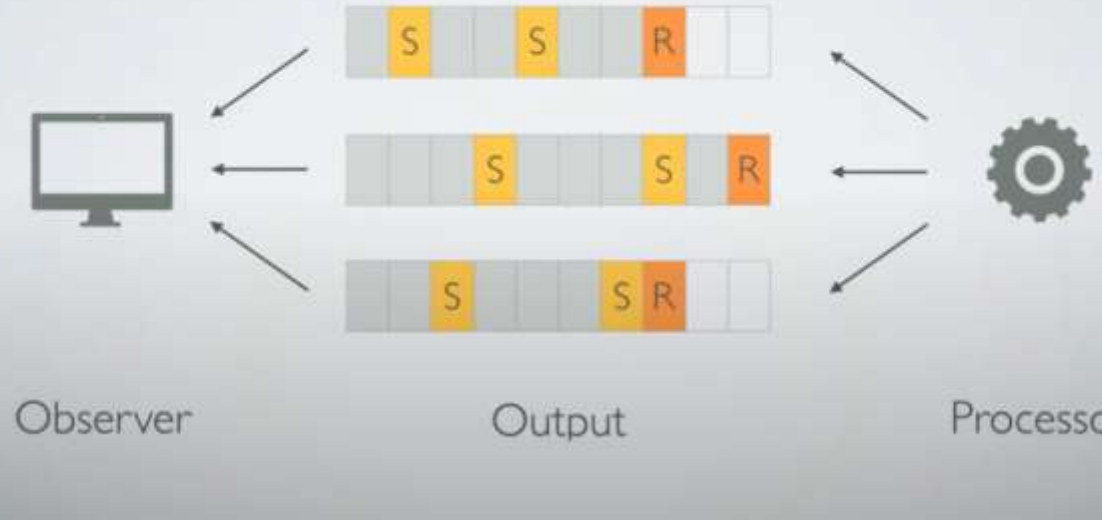
CHANDY-LAMPORT SNAPSHOTS

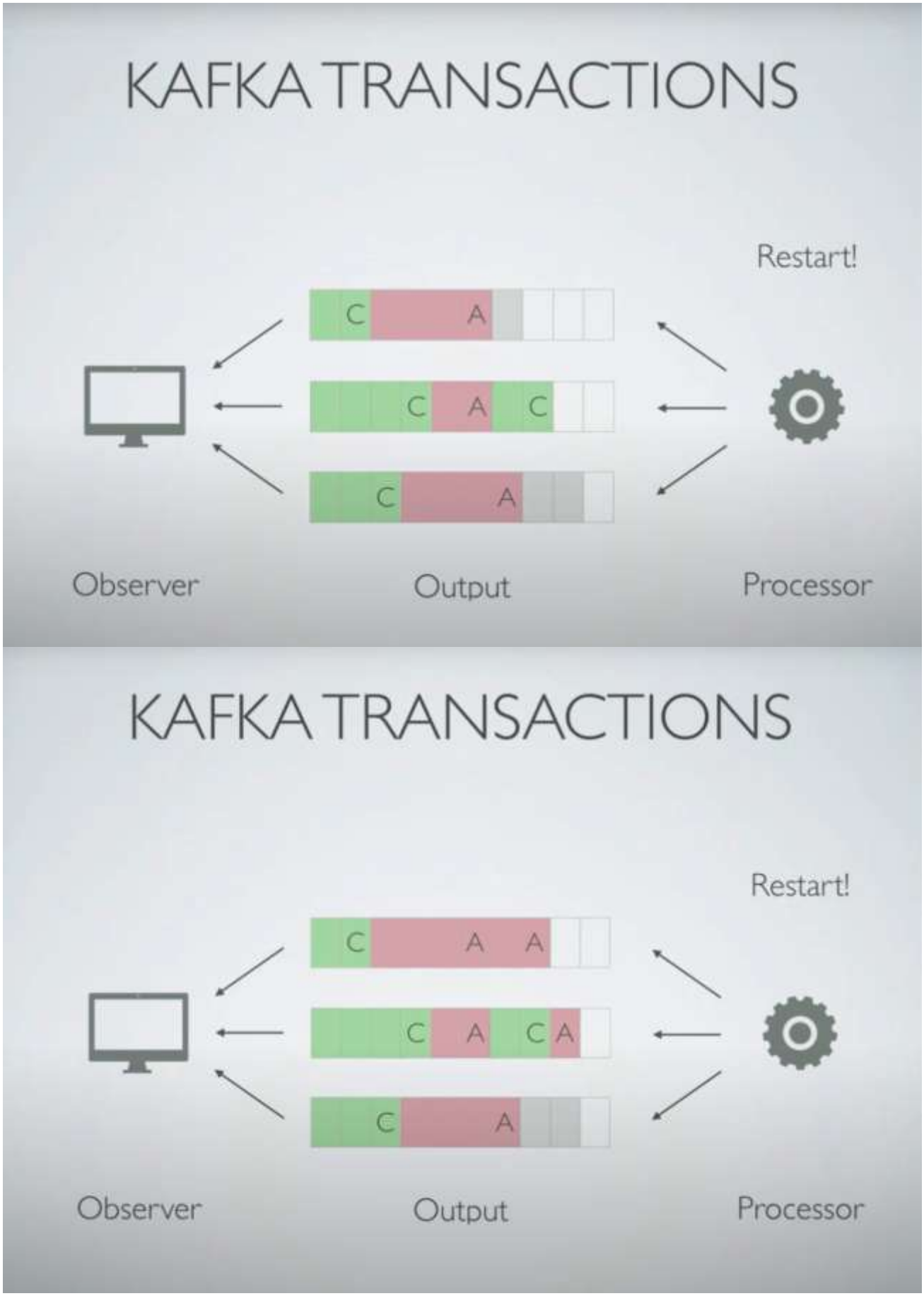


CHANDY-LAMPORT SNAPSHOTS



CHANDY-LAMPORT SNAPSHOTS





Commit of 2nd Partition is done, Observer will come on this and wait for all partition to commit.

KAFKA TRANSACTIONS

- Two-phase commit protocol with transaction log
  - Write intent to commit/abort to log
  - Write markers to data topics
  - Write completion of commit/abort to log

KAFKA TRANSACTIONS

Transaction Log

Partitions	(0, 1, 2)	(0, 1, 2)	(0, 1, 2)	(0, 1)	(0, 1)	**(0, 1)**
State	ongoing	prepare commit	complete commit	ongoing	prepare abort	**complete abort**

SEMANTIC WEAKNESSES

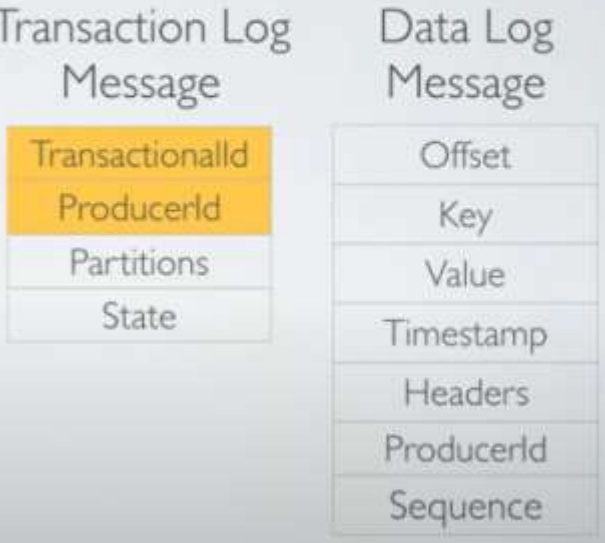
- ~~Producer retries are not safe~~
- ~~Processed data is not written atomically with corresponding offsets~~
- No protection from evil zombies

ZOMBIE FENCING

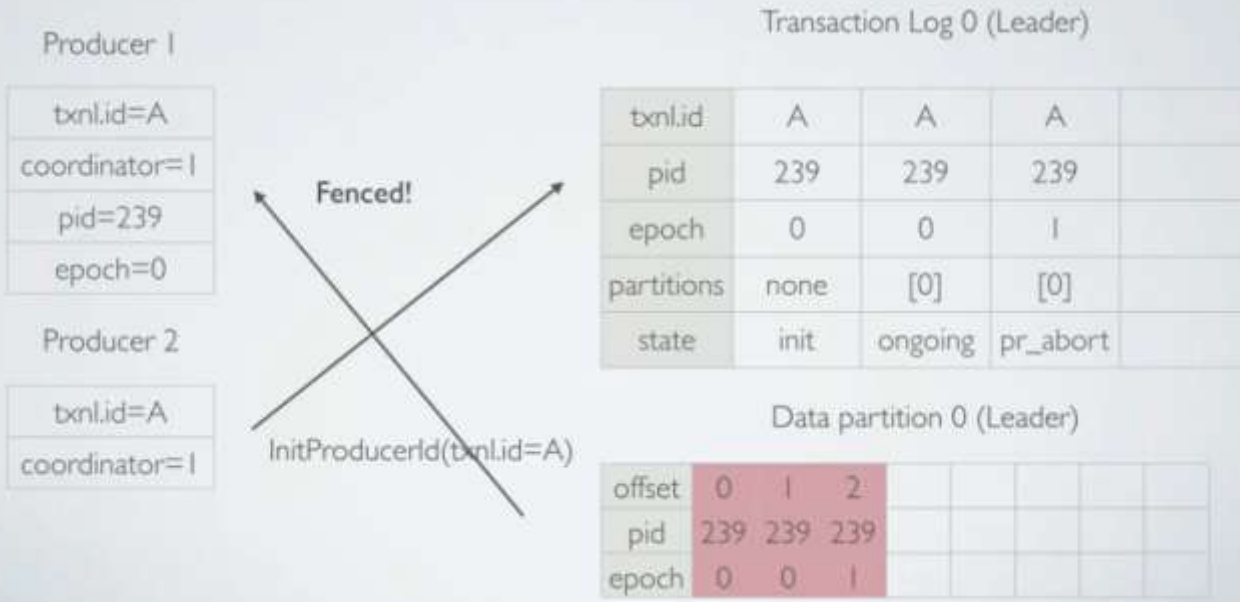
- To stop zombies, we need a fence
- But before that, we need a way to identify producer instances across restarts

ZOMBIE FENCING

- A "transactionalId" maps to a producerId
- So no need to persist the producerId
- transactional.id={id}



ZOMBIE FENCING



SEMANTIC WEAKNESSES

- ~~Producer retries are not safe~~
- ~~Processed data is not written atomically with corresponding offsets~~
- ~~No protection from evil zombies~~

EXACTLY-ONCE SEMANTICS

```
p.initTxns()
c.subscribe(inputTopics)
while (true) {
  offsets, input = c.poll()
  output = process(input)
  p.beginTxn()
  p.send(output)
  p.sendOffsets(offsets)
  p.commitTxn()
}
```

```
void beginTransaction();
```

EXACTLY-ONCE SEMANTICS

```
p.initTxns()
c.subscribe(inputTopics)
while (true) {
  offsets, input = c.poll()
  output = process(input)
  p.beginTxn()
  p.send(output)
  p.sendOffsets(offsets)
  p.commitTxn()
}
```

```
void sendOffsetsToTransaction(
  Map<TopicPartition, OffsetAndMetadata> offsets,
  String consumerGroupId)
throws ProducerFencedException;
```

EXACTLY-ONCE SEMANTICS

```
p.initTxns()
c.subscribe(inputTopics)
while (true) {
  offsets, input = c.poll()
  output = process(input)
  p.beginTxn()
  p.send(output)
  p.sendOffsets(offsets)
  p.commitTxn()
}
```

```
void commitTransaction()
  throws ProducerFencedException;
```

EXACTLY-ONCE SEMANTICS

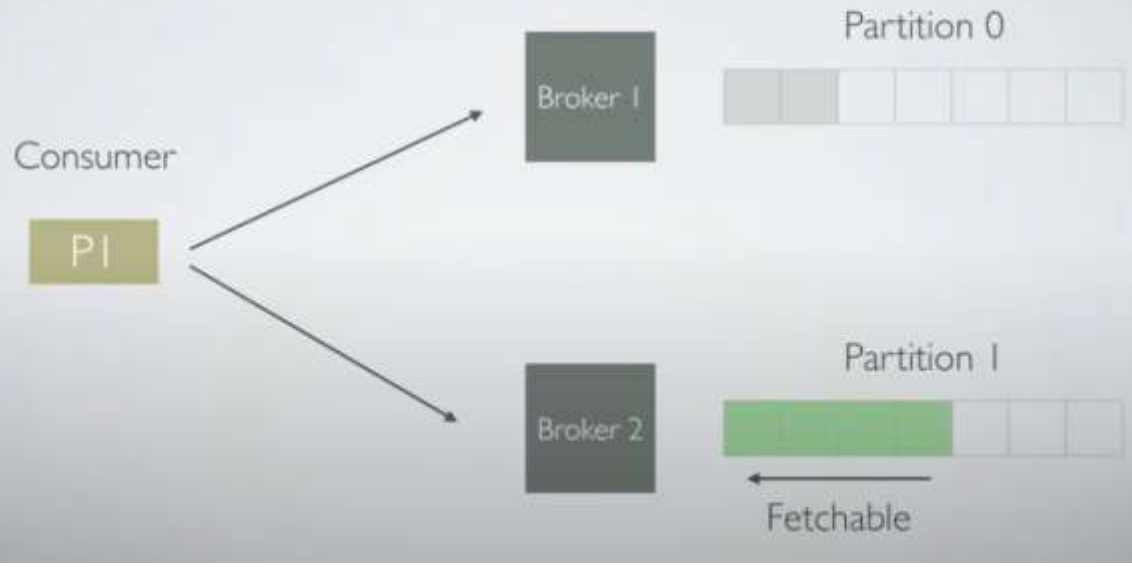

```
p.initTxns()
while (true) {
  offsets, data = c.poll()
  p.beginTxn()
  p.send(data)
  p.sendOffsets(offsets)
  p.commitTxn()
}
```

```
p.initTxns()
while (true) {
  offsets, data = c.poll()
  p.beginTxn()
  p.send(data)
  p.sendOffsets(offsets)
  p.commitTxn()
}
```

CONSUMER READ ISOLATION

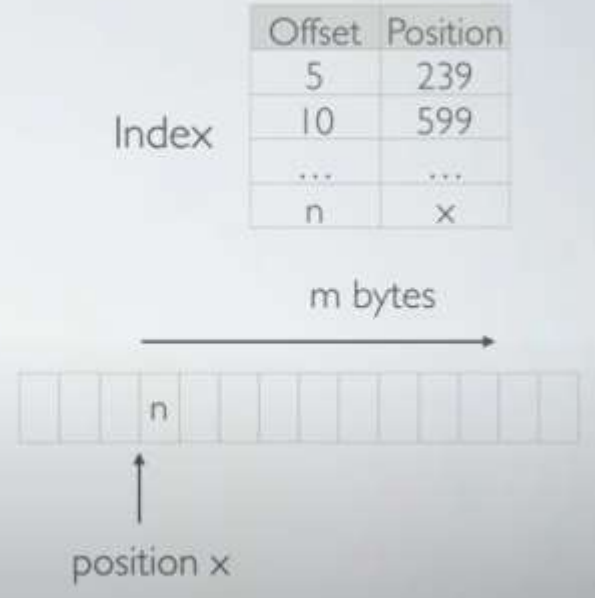
- Consumer **isolation.level**:
  - **read_committed**: only committed messages are readable
  - **read_uncommitted**: all messages are readable

CONSUMER READ ISOLATION



BROKER FETCH HANDLING

- Fetch(offset=n, bytes=m)
  - Lookup file position of offset n
  - Use sendfile to transfer m bytes from that position



BROKER FETCH HANDLING

- Fetch(offset=n, bytes=m)
  - Lookup file position of offset n
  - Use sendfile to transfer m bytes from that position

Aborted transactions not filtered!

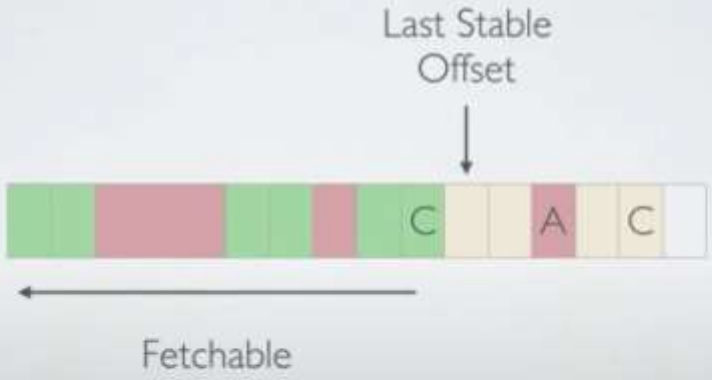
BROKER FETCH HANDLING

- The starting offsets of all aborted transactions are included in the fetch response
- Messages are not readable until they are committed or aborted

CONCURRENT TRANSACTIONS

Last Stable Offset: the first offset such that all prior offsets are committed or aborted

CONCURRENT TRANSACTIONS



Last Stable Offset: the first offset such that all prior offsets are committed or aborted

KAFKA EXACTLY-ONCE SEMANTICS

- Idempotent producer: exactly-once in-order delivery for a producer instance
- Transactional producer: atomic writes across multiple partitions (including consumer offsets)
- Read-committed consumer: only committed messages are returned

PERFORMANCE OBSERVATIONS

- Transaction overhead is minimal
- Overhead can be made arbitrarily small by increasing the size of the transaction, but…
- Longer transactions increase end-to-end latency

END-TO-END EOS

(Kafka Streams)

processing.guarantee=exactly_once

END-TO-END EOS